

Trabajo Práctico 1

TDAs: Conceptos Básicos

Definición, implementación y utilización de Pila, Cola y Cola con Prioridad

Algoritmos y Estructuras de Datos II — Grupo 2

Lenguaje: Java

Integrantes: Alejo Maggi · Mateo Galli · Juan Ventola · Francisco Medina · Ayelén Balmaceda

Fecha: 12/09 /2026

Parte 1 — TDA Pila

1.1 Especificación

Dominio: Pila = secuencia finita y ordenada de elementos, donde el último elemento en ingresar es el primero en salir (LIFO). Se implementa de forma estática, por lo que tiene una capacidad máxima fija.

Operaciones

crear() → Pila

Post: devuelve una pila vacía.

esVacia(p: Pila) → boolean

Post: devuelve true si p no tiene elementos, false en caso contrario.

esLlena(p: Pila) → boolean

Post: devuelve true si p alcanzó su capacidad máxima.

apilar(p: Pila, x: elemento) → Pila

Pre: p no está llena.

Post: x queda ubicado en el tope de p.

desapilar(p: Pila) → elemento

Pre: p no está vacía.

Post: se elimina el elemento del tope de p y se devuelve.

tope(p: Pila) → elemento

Pre: p no está vacía.

Post: devuelve el elemento del tope de p, sin eliminarlo.

1.2 Implementación — complejidad temporal

Se implementaron dos variantes sobre un arreglo de tamaño fijo, ambas cumpliendo la interfaz Pila.

Operación	Variante A (tope al final)	Variante B (tope en posición 0)
apilar	$O(1)$	$O(n)$
desapilar	$O(1)$	$O(n)$
tope	$O(1)$	$O(1)$

Justificación. En la Variante A el tope vive en el último lugar ocupado: apilar escribe una posición y avanza un índice, y desapilar hace lo inverso; ninguna operación recorre el arreglo, por eso todas son $O(1)$. En la Variante B el tope debe estar siempre en la posición 0: para apilar hay que desplazar todos los elementos un lugar hacia la derecha antes de insertar el nuevo tope, y para desapilar hay que desplazarlos hacia la izquierda; ese corrimiento es proporcional a la cantidad de elementos, por eso apilar y desapilar son $O(n)$. Solo tope es $O(1)$ en ambas.

¿Cuál conviene? ¿Depende de algo? Conviene la Variante A, y no depende del contexto de uso: iguala a la B en tope y la supera en apilar y desapilar, así que la domina en todas las operaciones. La Variante B solo tendría sentido si un requisito externo obligara a que el tope ocupe físicamente la posición 0, lo cual no ocurre en este TDA.

1.3 Utilización — complejidad de cada método

n = cantidad de elementos de la pila recibida.

Método	Complejidad	Observación
pasarPila	$O(n)$	una sola pasada; la pila devuelta queda en orden inverso
copiarPila	$O(n)$	dos pasadas; deja p en su estado original
invertirPila	$O(n)$	recursiva, profundidad n con $O(1)$ por nivel
masDeUnaOcurrencia	$O(n^2)$	por cada elemento se busca en el resto (búsqueda

Método	Complejidad	Observación
		$O(n)$
eliminarImpares	$O(n)$	dos pasadas; conserva el orden relativo de los pares

Parte 2 — TDA Cola

2.1 Especificación

Dominio: Cola = secuencia finita y ordenada de elementos, donde el primer elemento en ingresar es el primero en salir (FIFO). Implementación estática con capacidad máxima fija.

Operaciones

crear() → Cola

Post: devuelve una cola vacía.

esVacía(c: Cola) → boolean

Post: devuelve true si c no tiene elementos, false en caso contrario.

esLlena(c: Cola) → boolean

Post: devuelve true si c alcanzó su capacidad máxima.

encolar(c: Cola, x: elemento) → Cola

Pre: c no está llena.

Post: x queda agregado al final de c.

desencolar(c: Cola) → elemento

Pre: c no está vacía.

Post: se elimina el elemento del frente de c y se devuelve.

frente(c: Cola) → elemento

Pre: c no está vacía.

Post: devuelve el elemento del frente de c, sin eliminarlo.

2.2 Implementación — complejidad temporal

Se implementaron dos variantes sobre un arreglo de tamaño fijo.

Operación	Variante A (lineal simple)	Variante B (circular)
encolar	$O(1)$	$O(1)$
desencolar	$O(1)$	$O(1)$
frente	$O(1)$	$O(1)$

Justificación. Cada operación individual es $O(1)$ en las dos variantes: encolar escribe en la posición de fin y desencolar lee la de inicio, avanzando un índice, sin recorrer el arreglo. La diferencia no está en el costo por operación sino en el uso del espacio a lo largo del tiempo. En la variante lineal el índice fin solo avanza y nunca retrocede: los lugares que quedan libres al desencolar (al principio del arreglo) se pierden de forma irreversible, y la cola puede marcarse “llena” aunque tenga pocos elementos vivos. La variante circular usa aritmética modular para que los índices “den la vuelta”, de modo que todo lugar liberado por un desencolar vuelve a estar disponible.

¿Cuál conviene? Conviene la Variante B (circular): con el mismo costo por operación, aprovecha toda la capacidad del arreglo en lugar de desperdiciarla. La lineal simple solo sería aceptable en usos de un único llenado y vaciado, sin reutilización.

2.3 Utilización — complejidad de cada método

n = cantidad de elementos de la cola recibida; k = parámetro del método `finalCoincide`.

Método	Complejidad	Observación
pasarCola	$O(n)$	una pasada; mantiene el orden y deja origen vacía
invertirColaConPila	$O(n)$	usa una Pila auxiliar; consume la cola de entrada
invertirColaSinPila	$O(n)$	recursión pura, profundidad n , sin estructura auxiliar
finalCoincide	$O(n)$	dos pasadas por cola para copiar y restaurar; comparación $O(k)$

Parte 3 — TDA Cola con Prioridad

3.1 Especificación

Dominio: Cola con Prioridad = colección finita de elementos, cada uno asociado a una prioridad (número entero). La operación de extracción devuelve siempre el elemento de mayor prioridad, sin importar el orden de inserción. Implementación estática con capacidad máxima fija.

Operaciones

crear() → ColaPrioridad

Post: devuelve una cola con prioridad vacía.

esVacia(cp: ColaPrioridad) → boolean

Post: devuelve true si cp no tiene elementos.

esLlena(cp: ColaPrioridad) → boolean

Post: devuelve true si cp alcanzó su capacidad máxima.

insertar(cp: ColaPrioridad, x: elemento, prio: entero) → ColaPrioridad

Pre: cp no está llena.

Post: x queda incorporado a cp con prioridad prio.

verMax(cp: ColaPrioridad) → elemento

Pre: cp no está vacía.

Post: devuelve el elemento de mayor prioridad, sin eliminarlo; ante empate, el que fue insertado primero.

extraerMax(cp: ColaPrioridad) → elemento

Pre: cp no está vacía.

Post: elimina y devuelve el elemento de mayor prioridad; ante empate, el que fue insertado primero.

Criterio de desempate. Cuando dos elementos tienen la misma prioridad, sale primero el que fue insertado primero (FIFO entre iguales). Este criterio es necesario para que el método combinar (10) pueda conservar el orden relativo de inserción entre elementos de igual prioridad.

Nota de diseño. extraerMax y verMax devuelven el par (elemento, prioridad) —no solo el elemento— porque el método de utilización combinar (3.3) necesita conocer la prioridad de cada elemento extraído para reinsertarlo correctamente en la cola resultado, sin poder asumir nada sobre la implementación interna de ninguna variante.

3.2 Implementación — complejidad esperada

Cotas esperadas de las dos variantes, respetando el criterio de desempate. A confirmar contra la implementación.

Operación	Variante A (desordenada)	Variante B (ordenada)
insertar	$O(1)$	$O(n)$
extraerMax	$O(n)$	$O(1)$
verMax	$O(n)$	$O(1)$

¿Cuál conviene? ¿Depende del contexto? Acá la elección depende del uso: ninguna variante domina a la otra. La desordenada inserta en $O(1)$ pero paga $O(n)$ al extraer o consultar el máximo (debe recorrer todo el arreglo); la

ordenada accede al máximo en $O(1)$ pero paga $O(n)$ al insertar (debe ubicar el elemento en su lugar). Conviene la desordenada cuando predominan las inserciones, y la ordenada cuando predominan las extracciones o consultas del máximo.

3.3 Utilización — complejidad de cada método

n = cantidad de elementos de la cola con prioridad recibida (n_1 y n_2 para combinar).

Método	Complejidad	Observación
combinar	$O(n^2)$	vacía cp1 y cp2 con extraerMax y reinserta todo en el resultado con insertar; ambas colas de entrada quedan vacías
invertirColaConColaPrioridad	$O(n^2)$	vacía c asignando como prioridad el orden de llegada; extraerMax de la auxiliar entrega los elementos en orden inverso
sumarValoresPrioridadPar	$O(n^2)$	vacía cp acumulando los valores de prioridad par, y la reconstruye reinsertando en el mismo orden en que fue extraída